

Module	Système d'exploitation : architectures et outils
Enseignant	Hamza HAMMAMI
Année universitaire	2025-2026
École Nationale d'Ingénieurs de Tunis	

Programmation Shell avec Bash

Objectif du TP

L'objectif de ce TP est de familiariser les étudiants avec la programmation shell en utilisant le langage Bash. Ils apprendront à écrire des scripts Bash pour automatiser des tâches courantes, à manipuler des fichiers et des répertoires, ainsi qu'à utiliser des commandes système pour interagir avec le système d'exploitation.

Pré-requis

Pour suivre ce TP, les étudiants doivent avoir :

- Un environnement Linux ou Unix.
- Des connaissances de base en ligne de commande.

Les concepts clés :

- **Bash** : Le Bourne Again Shell est un interpréteur de commandes Unix et un langage de script.
- **Scripts Bash** : Des fichiers contenant des commandes Bash, utilisés pour automatiser des tâches.
- **Variables** : Des symboles utilisés pour stocker des valeurs.
- **Conditions et boucles** : Des structures de contrôle pour exécuter des commandes en fonction de conditions spécifiques.
- **Commandes système** : Des commandes pour interagir avec le système d'exploitation, telles que `ls`, `mkdir`, `cp`, etc.

1. Notions de base sur les variables

Variables dans le Shell

Une variable est utilisée pour stocker une valeur temporairement. La syntaxe de déclaration est :

```
NOM_VARIABLE="valeur"
```

Pour accéder à la valeur d'une variable, on utilise `$NOM_VARIABLE`.

Exemple d'utilisation :

```
nom="Alice"
echo "Bonjour, _$nom!" # Affiche : Bonjour, Alice !
```

Règles pour les variables :

- Elles ne doivent pas contenir d'espaces.
- Elles ne peuvent pas commencer par un chiffre.
- Les valeurs sont sensibles à la casse : `VAR` et `var` sont des variables différentes.

Liste de variables prépositionnées

- `$0` : nom du script. Plus précisément, il s'agit du paramètre 0 de la ligne de commande.
- `$1`, `$2`, ..., `$9` : respectivement premier, deuxième, ..., neuvième paramètre de la ligne de commande.
- `$*` : tous les paramètres vus comme un seul mot.
- `$@` : tous les paramètres vus comme des mots séparés : `"$@"` équivaut à `"$1" "$2" ...`
- `$` : nombre de paramètres sur la ligne de commande.
- `$?` : état de sortie de la dernière commande exécutée.

Exemple

Écrire et exécuter le code suivant dans un fichier `votre_nom.sh` :

```
#!/bin/bash
# script1.sh
echo "Nom _du _script _$0"
echo "premier _paramètre _$1"
echo "second _paramètre _$2"
echo "nombre _de _paramètres _$#"
var=bonjour
echo "Il _y _a _${#var} _caractères"
```

2. Commande read et echo

read : Lire des entrées utilisateur

La commande `read` permet de demander une entrée utilisateur.

Syntaxe basique :

```
read -p "Entrez _votre _prénom : _" prenom
echo "Bonjour, _$prenom!"
```

Autres possibilités avec read :

- `read var1 var2` : Lire plusieurs valeurs d'une seule ligne.
- `read -s` : Masquer l'entrée (utile pour les mots de passe).
- `read -n 1 var` : Lire uniquement un caractère.
- `read -t 5 var` : Lire une entrée avec un délai de 5 secondes.

echo : Afficher du texte

- `echo "Texte"` : Affiche un texte simple.
- `echo -e "Ligne 12"` : Active les caractères spéciaux comme `\n`.
- `echo -n "Texte"` : Affiche sans sauter de ligne.

Exemple complet combinant read et echo :

```
read -p "Entrez votre prénom : " prenom
read -s -p "Entrez votre mot de passe : " mdp
echo -e "\nBonjour, $prenom. Votre mot de passe a été
    enregistré."
```

Autres exemples combinant les commandes read et echo

1. Lire plusieurs valeurs d'une seule ligne

Script :

```
#!/bin/bash
echo "Entrez deux valeurs séparées par un espace : "
read var1 var2
echo "Première valeur : $var1"
echo "Deuxième valeur : $var2"
```

Sortie :

```
Entrez deux valeurs séparées par un espace :
Bonjour Monde
Première valeur : Bonjour
Deuxième valeur : Monde
```

2. Masquer l'entrée (utile pour les mots de passe)

Script :

```
#!/bin/bash
read -s -p "Entrez votre mot de passe : " password
echo
echo "Mot de passe masqué entré."
```

Sortie :

```
Entrez votre mot de passe :  
Mot de passe masqué entré.
```

3. Lire uniquement un caractère

Script :

```
#!/bin/bash  
read -n 1 -p "Appuyez sur une touche pour continuer : "  
    touche  
echo  
echo "Vous avez appuyé sur la touche : $touche"
```

Sortie :

```
Appuyez sur une touche pour continuer : a  
Vous avez appuyé sur la touche : a
```

4. Lire une entrée avec un délai de 5 secondes

Script :

```
#!/bin/bash  
read -t 5 -p "Entrez quelque chose (vous avez 5 secondes) : "  
    input  
if [ -z "$input" ]; then  
    echo "Temps écoulé sans entrée."  
else  
    echo "Vous avez entré : $input"  
fi
```

Sortie :

```
Entrez quelque chose (vous avez 5 secondes) :  
Temps écoulé sans entrée.  
-- ou --  
Entrez quelque chose (vous avez 5 secondes) : Hello  
Vous avez entré : Hello
```

3. Opérateurs de Comparaison et Autres Commandes

Opérateurs de Comparaison d'Entiers

En bash, les opérateurs de comparaison d'entiers sont utilisés avec la commande `test` ou les crochets `[ ]`. Voici les opérateurs les plus utilisés pour comparer des entiers :

— `-eq` : égal à.

- **-ne** : différent de.
- **-lt** : inférieur à.
- **-le** : inférieur ou égal à.
- **-gt** : supérieur à.
- **-ge** : supérieur ou égal à.

Exemple d'utilisation pour la comparaison d'entiers :

```
a=5
b=10
if [ $a -lt $b ]; then
    echo "$a est inférieur à $b"
fi
```

Dans cet exemple, a est inférieur à b, donc le message sera affiché.

Opérateurs de Comparaison sur les Chaînes de Caractères

Les opérateurs de comparaison pour les chaînes de caractères en bash permettent de tester des relations entre deux chaînes.

- **=** : égal à.
- **!=** : différent de.
- **-z** : la chaîne est vide.
- **-n** : la chaîne n'est pas vide.
- **<** : lexicographiquement avant.
- **>** : lexicographiquement après.

Exemple de comparaison de chaînes :

```
str1="Bonjour"
str2="Salut"
if [ "$str1" != "$str2" ]; then
    echo "Les chaînes sont différentes."
fi
```

Dans cet exemple, les chaînes `str1` et `str2` sont différentes, donc le message sera affiché.

Opérateurs de Comparaison sur les Fichiers

En bash, il existe plusieurs opérateurs pour tester des propriétés d'un fichier.

- **-e** : Vérifie si le fichier existe.
- **-f** : Vérifie si c'est un fichier ordinaire.
- **-d** : Vérifie si c'est un répertoire.
- **-s** : Vérifie si le fichier n'est pas vide.
- **-r** : Vérifie si le fichier est lisible.
- **-w** : Vérifie si le fichier est modifiable.
- **-x** : Vérifie si le fichier est exécutable.

Exemple de comparaison sur un fichier :

```
fichier="exemple.txt"
if [ -f "$fichier" ]; then
    echo "$fichier est un fichier ordinaire."
fi
```

Si `exemple.txt` est un fichier ordinaire, le message sera affiché.

Opérations sur les Nombres : Addition, Soustraction, Multiplication, etc.

Voici les opérations arithmétiques de base en bash :

- Addition : +
- Soustraction : -
- Multiplication : *
- Division : /
- Puissance : **
- Modulo (reste de la division) : %

Exemple d'opérations arithmétiques :

```
a=5
b=3
sum=$((a + b))
echo "La somme de $a et $b est $sum"
```

Cet exemple affiche la somme de `a` et `b`, c'est-à-dire 8.

Formatage des Dates et Heures

Voici comment afficher la date et l'heure dans différents formats en utilisant la commande `date` :

- `%H` : L'heure au format 00..23.
- `%M` : Minutes 00..59.
- `%S` : Secondes 00..60.
- `%A` : Jour complet.
- `%B` : Mois complet.
- `%d` : Jour du mois.
- `%m` : Numéro du mois.
- `%y` : Deux derniers chiffres de l'année.
- `%Y` : Année complète.

Exemple d'affichage de la date :

```
date -d "+1 days" +"%A %d/%m/%Y"
```

Cet exemple affiche la date du jour suivant dans le format complet :
 Jour mois/numéro_du_mois/année.

Les Quotes (Guillemets)

Sous bash, vous pouvez utiliser deux types de quotes pour délimiter des chaînes :

- " " : Double quotes ou guillemets. Utilisés pour autoriser l'expansion de variables à l'intérieur de la chaîne.
- ' ' : Simple quotes. Utilisés pour rendre la chaîne littérale (aucune expansion de variable).

Exemple avec les double quotes :

```
nom="Alice"
echo "Bonjour, □$nom□!" # Affiche : Bonjour, Alice !
```

Exemple avec les simples quotes :

```
echo 'Bonjour, □$nom' # Affiche : Bonjour, $nom
```

Dans ce dernier exemple, les simples quotes empêchent l'expansion de la variable \$nom.

4. Structures conditionnelles

Syntaxe if

```
if [ condition ]; then
    # Commandes si la condition est vraie
elif [ autre_condition ]; then
    # Commandes si l'autre condition est vraie
else
    # Commandes si aucune condition n'est vraie
fi
```

Exemple : Vérifier l'âge d'une personne

```
read -p "Entrez □votre □ge □:□" age
if [ $age -lt 18 ]; then
    echo "Vous □êtes □mineur."
elif [ $age -eq 18 ]; then
    echo "Vous □avez □exactement □18 □ans."
else
    echo "Vous □êtes □majeur."
fi
```

5. Boucles en Bash

Boucle while

```
compteur=1
while [ $compteur -le 5 ]; do
    echo "Compteur : $compteur"
    compteur=$((compteur + 1))
done
```

Boucle for

```
for fichier in *.txt; do
    echo "Nom du fichier : $fichier"
done
```

6. Exercices

Exercice 1 : Compter les fichiers dans un répertoire

- **Énoncé :** Créer un script qui affiche le nombre total de fichiers présents dans le répertoire actuel.
- **Explication pour les étudiants :**
 - Vous devez utiliser la commande Linux `ls` pour lister les fichiers du répertoire.
 - Utilisez la commande `wc -l` pour compter le nombre de lignes, car chaque fichier correspond à une ligne dans la sortie de `ls`.
 - Affichez le résultat à l'utilisateur avec `echo`.
- **Étapes guidées :**
 - Ouvrir un éditeur de texte comme `nano` pour écrire votre script.
 - Sauvegarder votre script avec une extension `.sh`.
 - Donner les permissions d'exécution avec la commande :

```
chmod +x votre_script.sh
```


Exercice 2 : Afficher les fichiers cachés selon la demande de l'utilisateur

- **Énoncé** : Créer un script Shell qui demande à l'utilisateur s'il souhaite afficher ou non les fichiers cachés du répertoire actuel.
- **Étapes guidées** :
 - Les fichiers cachés sous Linux commencent par un point (.).
 - Utilisez la commande `ls -a` pour afficher tous les fichiers (y compris les fichiers cachés).
 - Demandez à l'utilisateur de répondre par "o" (oui) ou "n" (non).
 - Si l'utilisateur répond "o", affichez tous les fichiers. Sinon, affichez uniquement les fichiers visibles.
- **Étapes guidées** :
 - Utilisez la commande `read` pour lire la réponse de l'utilisateur.
 - Les structures conditionnelles comme `if` seront nécessaires pour effectuer des choix basés sur la réponse de l'utilisateur.

Exercice 3 : Création d'un répertoire avec des fichiers

- **Énoncé** : Créer un script qui :
 - Crée un répertoire nommé `Ex1`.
 - Dans ce répertoire, crée cinq fichiers nommés respectivement `Un`, `Deux`, `Trois`, `Quatre`, et `Cinq`.
 - Chaque fichier doit contenir une seule ligne de texte :
 - Le fichier `Un` contient `Première ligne`.
 - Le fichier `Deux` contient `Deuxième ligne`.
 - Le fichier `Trois` contient `Troisième ligne`.
 - Le fichier `Quatre` contient `Quatrième ligne`.
 - Le fichier `Cinq` contient `Cinquième ligne`.
- **Étapes guidées** :
 - Utilisez la commande `mkdir` pour créer un répertoire.
 - Utilisez la commande `echo` pour écrire une ligne dans un fichier, combinée à la redirection `>`.
 - Vous pouvez automatiser la création des fichiers avec une boucle pour éviter de répéter la commande.
- **Concept clé à comprendre** : La commande `echo` envoie du texte vers la sortie standard ou vers un fichier. La redirection `>` permet de sauvegarder ce texte dans un fichier.

Exercice 4 : Comparaison de trois chaînes de caractères

- **Énoncé** : Créer un script Shell qui :
 - Lit trois chaînes de caractères :
 - Soit en ligne de commande (comme arguments du script).
 - Soit au clavier (si aucun argument n'est donné).
 - Compare les trois chaînes et affiche un message indiquant :
 - Si les trois chaînes sont identiques.
 - Si seulement deux d'entre elles sont identiques.
 - Si les trois chaînes sont différentes.
- **Étapes guidées** :
 - Utilisez des variables comme \$1, \$2, \$3 pour lire les arguments passés en ligne de commande.
 - Sinon, utilisez la commande `read` pour demander les chaînes à l'utilisateur.
 - Utilisez des conditions `if` et des opérateurs de comparaison `=` pour vérifier si deux chaînes sont identiques.
- **Concept clé à comprendre** : En `bash`, utilisez `[ "$chaine1" = "$chaine2" ]` pour comparer deux chaînes de caractères.

Exercice 5 : Vérifier l'existence d'un utilisateur

- **Énoncé** : Créer un script qui vérifie si un utilisateur donné existe sur le système. L'utilisateur sera passé comme paramètre au script.
- **Étapes guidées** :
 - Utilisez la commande `id` pour vérifier si un utilisateur existe.
 - Si l'utilisateur existe, la commande retourne des informations sur cet utilisateur. Sinon, une erreur sera générée.
 - Affichez un message indiquant si l'utilisateur existe ou non.
- **Étapes guidées** : Utilisez des redirections pour gérer les sorties des commandes : `>/dev/null` permet de cacher la sortie d'une commande.

Exercice 6 : Tester un fichier passé en paramètre

- **Énoncé :** Créer un script qui effectue les tests suivants sur un fichier passé en argument au script :
 - Vérifie si le fichier existe.
 - Affiche le type de fichier (fichier ordinaire ou répertoire).
 - Vérifie si le fichier est non vide.
 - Affiche les droits d'accès du fichier (lecture, écriture, exécution).
- **Étapes guidées :**
 - Utilisez l'option `-e` avec la commande `test` pour vérifier si un fichier existe.
 - Utilisez `-f` pour vérifier s'il s'agit d'un fichier ordinaire.
 - Utilisez `-d` pour vérifier s'il s'agit d'un répertoire.
 - Testez si le fichier est vide avec `-s`.
 - Vérifiez les permissions avec les options `-r`, `-w`, et `-x`.
- **Concept clé à comprendre :** Les fichiers et répertoires sous Linux possèdent des permissions associées. Les tests conditionnels du Shell permettent de vérifier ces propriétés facilement.

À l'issue de ce TP, les étudiants seront capables de maîtriser :

- La création et l'exécution de scripts Shell en utilisant `bash`.
- La manipulation de variables, chaînes de caractères, et fichiers sous Linux.
- L'utilisation de structures de contrôle comme les conditions (`if`, `elif`, `else`) et les boucles (`for`, `while`).
- L'application de commandes Linux dans un script pour automatiser des tâches.